# Laboratorio 2 — Semáforos en XV6

## Clase 5: post-intermedia y arranque de prodcons

Sistemas Operativos — FaMAF, UNC

29/09/2026

# Agenda

- Dónde estamos
- Retro de la intermedia
- Productor–consumidor
- `prodcons B N`
- Camino a la final

# Dónde estamos

# Calendario

| Fecha | Evento |
|---|---|
| 15/09 – 24/09 | Clases 1–4: xv6, syscalls, `pingpong` |
| **28/09** | Intermedia **11/11** (cerrada) |
| **29/09** | **Hoy** — arranque de `prodcons` |
| 01/10 | Clase 6 — implementación de `prodcons` |
| 06/10 | Clase 7 — cierre de la final + video |
| **07/10** | Final **15/15** + video (23:59) |

# De 11 a 15 puntos

| Bloque | Tests | Puntos |
|---|---|---|
| `semtest` | noinit, init, closeuse, exhaust, grow, reopen | 6 |
| `pingpong` | args, orden, concurrencia | 5 |
| `prodcons` | sin args, args inválidos, B=1, B=3 | 4 |

`make grade-parcial` → **11/11**

`make grade-final` → meta **15/15** (+ video)

# Retro de la intermedia

# Errores que vimos

| **Síntoma** | **Causa** |
|---|---|
| `pingpong` se cuelga | valores iniciales invertidos; `sem_down` sin `sem_up` |
| falla `semtest grow` | `sem_up` topeaba en el valor inicial |
| falla `semtest closeuse` | `sem_close` sin wakeup; los bloqueados no devuelven 0 |
| `pingpong` OK pero score $< 11$ | casi siempre falla algún test de `semtest` |

# El kernel en una slide

```
sem_open   ids 1..64, tabla global, falla si ya está abierto
sem_close  marca cerrado + wakeup
sem_down   while (value == 0) sleep;   value--
sem_up     value++;  wakeup            (sin tope, salvo SEM_VALUE_MAX)
```

`prodcons` no toca el kernel: usa las mismas cuatro syscalls. Si alguien no cerró la intermedia, consulta fuera de clase.

# Productor–consumidor

# El problema

- Un proceso produce ítems; otro los consume.
- Entre los dos, un buffer de capacidad **B**.
- Si el buffer está **lleno**, el productor espera.
- Si está **vacío**, el consumidor espera.

## Invariantes

$0 \leq$ ítems en el buffer $\leq B$

No se consume un ítem que todavía no se produjo.

# Frente a pingpong

| | `pingpong` | `prodcons` |
|---|---|---|
| Sincronización | alternancia estricta | ritmos desacoplados |
| Semáforos | binarios (1 y 0) | contadores (B y 0) |
| Adelantarse | nunca | hasta B ítems |

- Con $\mathbf{B = 1}$, `prodcons` se comporta como `pingpong`.
- Con $\mathbf{B > 1}$, el productor puede ir varios pasos adelante: como un pipe de Unix (`cmd1 | cmd2`).

# Dos semáforos contadores

| Semáforo | Arranca en | Cuenta |
|---|---|---|
| `empty` | **B** | lugares libres en el buffer |
| `full` | **0** | ítems listos para consumir |

El productor gasta un lugar libre y suma un ítem.
El consumidor gasta un ítem y devuelve un lugar.

Por eso `sem_up` no puede topear: `full` arranca en 0 y tiene que llegar a B.

# El protocolo

```
productor:   down(empty)  →  produce  →  up(full)
consumidor:  down(full)   →  consume  →  up(empty)
```

- Igual que en `pingpong`: cada uno baja el suyo y sube el del otro.
- La diferencia es que ahora hay **hasta B fichas** circulando, no una sola.

# Paso a paso (B = 3)

| **Qué pasa** | `empty` | `full` | **En vuelo** |
|---|---|---|---|
| estado inicial | 3 | 0 | 0 |

# Paso a paso (B = 3)

| **Qué pasa** | `empty` | `full` | **En vuelo** |
|---|---|---|---|
| estado inicial | 3 | 0 | 0 |
| P produce 1, 2, 3 | 0 | 3 | **3** |

# Paso a paso (B = 3)

| **Qué pasa** | `empty` | `full` | **En vuelo** |
|---|---|---|---|
| estado inicial | 3 | 0 | 0 |
| P produce 1, 2, 3 | 0 | 3 | **3** |
| P quiere producir 4 | 0 | 3 | bloqueado en `empty` |

# Paso a paso (B = 3)

| **Qué pasa** | `empty` | `full` | **En vuelo** |
|---|---|---|---|
| estado inicial | 3 | 0 | 0 |
| P produce 1, 2, 3 | 0 | 3 | **3** |
| P quiere producir 4 | 0 | 3 | bloqueado en `empty` |
| C consume | 1 | 2 | 2 |

# Paso a paso (B = 3)

| **Qué pasa** | `empty` | `full` | **En vuelo** |
|---|---|---|---|
| estado inicial | 3 | 0 | 0 |
| P produce 1, 2, 3 | 0 | 3 | **3** |
| P quiere producir 4 | 0 | 3 | bloqueado en `empty` |
| C consume | 1 | 2 | 2 |
| P despierta, produce 4 | 0 | 3 | 3 |

# Paso a paso (B = 3)

| **Qué pasa** | `empty` | `full` | **En vuelo** |
|---|---|---|---|
| estado inicial | 3 | 0 | 0 |
| P produce 1, 2, 3 | 0 | 3 | **3** |
| P quiere producir 4 | 0 | 3 | bloqueado en `empty` |
| C consume | 1 | 2 | 2 |
| P despierta, produce 4 | 0 | 3 | 3 |
| C consume todo | 3 | 0 | 0 |

# Paso a paso (B = 3)

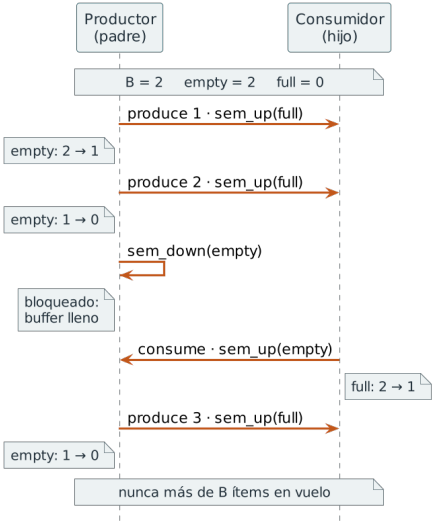
| **Qué pasa** | `empty` | `full` | **En vuelo** |
|---|---|---|---|
| estado inicial | 3 | 0 | 0 |
| P produce 1, 2, 3 | 0 | 3 | **3** |
| P quiere producir 4 | 0 | 3 | bloqueado en `empty` |
| C consume | 1 | 2 | 2 |
| P despierta, produce 4 | 0 | 3 | 3 |
| C consume todo | 3 | 0 | 0 |
| C quiere consumir | 3 | 0 | bloqueado en `full` |

# Paso a paso (B = 3)

| **Qué pasa** | `empty` | `full` | **En vuelo** |
|---|---|---|---|
| estado inicial | 3 | 0 | 0 |
| P produce 1, 2, 3 | 0 | 3 | **3** |
| P quiere producir 4 | 0 | 3 | bloqueado en `empty` |
| C consume | 1 | 2 | 2 |
| P despierta, produce 4 | 0 | 3 | 3 |
| C consume todo | 3 | 0 | 0 |
| C quiere consumir | 3 | 0 | bloqueado en `full` |

`empty` + `full` = B, salvo en el instante entre un down y su up.

# Cómo se ve en el tiempo (B = 2)



# ¿Dónde está el buffer?

Después de `fork`, padre e hijo no comparten memoria de usuario.

- Un `int buffer[B]` en el padre y otro en el hijo son **dos arreglos distintos**.
- En este lab el buffer es lógico: lo que importa es cuántos ítems hay "en vuelo", y eso lo cuentan los semáforos.
- La evidencia es la **salida**: las líneas `produce` / `consume`.

# El tercer semáforo: `mutex`

- El `printf` de xv6 hace un `write` por carácter.
- Si productor y consumidor imprimen a la vez, las líneas se **mezclan**:

```
prcoondsuucmee  1p ipdi=d5=
4
```

`mutex` arranca en **1** y protege la impresión: una línea completa por vez. Con varios productores o consumidores (punto estrella) es imprescindible.

# El orden de los down importa

```
down(empty);  down(mutex);     // bien: primero el recurso

down(mutex);  down(empty);     // MAL
```

- Con el buffer lleno, el productor toma el `mutex` y se duerme en `empty` con el mutex tomado.
- El consumidor baja `full` y se bloquea en `mutex`: nunca llega a subir `empty`.
- Deadlock: cada uno espera algo que tiene el otro.

# prodcons B N

# Interfaz

```
$ prodcons
ERROR: uso: prodcons B N  |  prodcons B N P C

$ prodcons 0 5
ERROR: B y N deben ser enteros >= 1
```

- **B**: capacidad del buffer. **N**: ítems a producir y a consumir.
- Los errores empiezan con `ERROR`, como en `pingpong`.
- Un solo `fork`: un productor (ítems $1..N$) y un consumidor ($N$ consumos).

# Qué tiene que salir

```
$ prodcons 1 3
produce 1 pid=4
consume pid=5
produce 2 pid=4
consume pid=5
produce 3 pid=4
consume pid=5
```

- Con $\mathbf{B = 1}$: alternado, obligatoriamente.
- Con $\mathbf{B = 2}$: puede haber dos `produce` seguidos, pero nunca tres ítems en vuelo.
- El **pid** muestra que son procesos distintos.

```
printf("produce %d pid=%d\n", k, getpid());
printf("consume pid=%d\n", getpid());
```

# Cómo lo corrige el grader

| Test | Qué mira |
|---|---|
| `prodcons` | sale ERROR |
| `prodcons 0 5` | sale ERROR |
| `prodcons 1 5` | 5 produce, 5 consume, nunca más de 1 en vuelo |
| `prodcons 3 10` | 10 y 10, nunca más de 3 en vuelo |

- Recorre la salida contando: `produce` suma 1, `consume` resta 1. El máximo no puede pasar de **B** y al final tiene que dar **0**.
- Además espera que el shell vuelva: el programa termina cuando terminan los dos procesos.

# Esqueleto

```
empty = sem_get(B);   full = sem_get(0);   mutex = sem_get(1);
if(fork() == 0){                    // hijo: consumidor
  for(i = 0; i < n; i++){
    sem_down(full);
    sem_down(mutex);
    printf("consume pid=%d\n", getpid());
    sem_up(mutex);
    sem_up(empty);
  }
  exit(0);
}
for(k = 1; k <= n; k++){            // padre: productor
  sem_down(empty);
  sem_down(mutex);
  printf("produce %d pid=%d\n", k, getpid());
  sem_up(mutex);
  sem_up(full);
}
wait(0);
sem_close(empty);  sem_close(full);  sem_close(mutex);
```

# Detalles del esqueleto

- `sem_get` es el mismo de `pingpong`: reserva el primer id libre. Nada de ids hardcodeados: el video pide correr `pingpong` y `prodcons` a la vez.
- El `printf` va **entre** el `down` y el `up`: así la salida refleja el orden real de los semáforos.
- `wait(0)` antes de cerrar: si el padre cierra con el hijo bloqueado, el `sem_down` del hijo devuelve 0.

# Errores típicos

| **Error** | **Qué pasa** |
|---|---|
| `full` arranca en B | el consumidor no espera: consume lo que no existe |
| `empty` arranca en 0 | deadlock desde el primer ítem |
| `mutex` antes del recurso | deadlock con el buffer lleno o vacío |
| `printf` fuera del par down/up | la salida viola la capacidad |
| Sin `wait` o sin `sem_close` | el shell vuelve antes / ids que no se liberan |

# Camino a la final

# Próximas clases

| Clase | Fecha | Tema |
|---|---|---|
| 6 | **01/10** | `prodcons`: implementación guiada + tests |
| 7 | **06/10** | `make grade-final`, video, punto estrella |

## Punto estrella (opcional): `prodcons B N P C`

Varios productores y consumidores con los mismos tres semáforos. Es `@CREDIT`: no suma al 15/15.

# Para el 01/10

1. `user/prodcons.c` en `UPROGS`, compilando.
2. Probar a mano: `prodcons 1 5` y `prodcons 4 20`.
3. Leer el enunciado: sección `prodcons` y guión del video.

`make grade-final` → meta **15/15**

Entrega final: **07/10 23:59**, con video.

# Para llevarse

`prodcons` reutiliza las mismas syscalls de la intermedia.

El salto es pensar en **contadores** (`empty`/`full`), no reescribir el kernel.

# Referencias

- **Enunciado** — *El problema productor-consumidor* y *El programa prodcons*.
- **OSTEP** — Semáforos (cap. 31), sección *The Producer/Consumer (Bounded Buffer) Problem*.